

Agentic Software Development

Day 1: using AI well

Tino Silfver · Jukka Loikkanen

Topcon Healthcare · June 2026



Two days, one arc

Day 1: use AI well

- M365 Copilot
- Prompting and grounding
- Copilot Studio agents
- Trust, verification, responsible use

Day 2: build with AI responsibly

- Spec-driven development
- GitHub Copilot
- Agentic development workflow
- Requirements to working code

Prerequisites, for both days

- Today: signed in to M365 with the Copilot Enterprise add-on (both tracks)
- Track B also needs Copilot Studio access
- Before Day 2: JDK 25, Git, VS Code with Copilot and the Java pack
- A GitHub account with a Copilot licence
- Clone the lab repo and run the build once, on your own network

◆ THE TWO THAT BITE

The licence blocks this afternoon; the first dependency download blocks Day 2. Do both before Tuesday.

Today

01

Introduction

09:00 · objectives and AI maturity check

02

M365 Copilot fundamentals

09:15 · grounding, permissions, prompting

03

Agents and responsible use

10:45 · Copilot Studio, etiquette, verification

04

Labs part 1, two tracks

13:00 · Office copilots and Copilot Studio agents

05

Labs part 2, quality and trust

14:45 · verify outputs, improve agents

06

Workshop and debrief

15:30 · use cases, risks, governance

AI maturity check

- I use Copilot or AI tools daily
- I use them sometimes
- I have tested them once or twice
- I am mostly watching carefully
- I am here to understand what is actually useful





01

Module 1: M365 Copilot

Fundamentals, grounding, permissions, prompting

What M365 Copilot is

- An AI assistant across Microsoft 365
- Summarize, draft, rewrite, compare, structure
- Reason over work content you can access
- Create first versions of work outputs

◆ ASSISTANT, NOT
AUTOPILOT

Copilot accelerates work but does not remove ownership. The first answer is a draft, not a verdict.

Copilot Chat versus M365 Copilot

Copilot Chat

- Good for general prompts
- Can work with uploaded files
- Drafting, reasoning, summarizing

M365 Copilot

- Uses Microsoft 365 work context
- Depends on licence and permissions
- Reasons over accessible work content

Microsoft Graph and grounding

- Answers are shaped by the prompt and available context
- Accessible files, meetings, chats and emails
- Permissions and source quality
- Two people, different access, different answers

◆ GROUNDED IN WHAT?

Copilot is like an analyst who can only read the folders they are allowed to open.

Security and permissions

- Copilot does not bypass permissions
- But it makes accessible information easier to find
- Overshared files become more visible
- Document hygiene and ownership matter more
- AI does not replace governance; it makes it visible

A simple prompt pattern

- Goal: what do you want?
- Context: what does Copilot need to know?
- Source: what should it use?
- Format: what should the output look like?
- Verification: what should it prove or mark uncertain?

Demos: Copilot on real work

- 1. Upload source
- 2. Ask for a useful output
- 3. Improve the format
- 4. Verify the answer
- 5. Decide what still needs human judgment



Coffee break

15 minutes · 10:25 to 10:40



02

Module 2: Agents and responsible use

Agents, etiquette, hallucinations, healthcare

What an AI agent is

- A chatbot answers; an agent works toward a goal
- A useful agent has purpose and instructions
- Knowledge, tools or actions, and boundaries
- Testing and a human owner

Copilot Studio in one slide

- Create agents with instructions and knowledge sources
- Skills or guidance, test chat, actions and integrations
- Governance controls
- Today: scope, knowledge, behavior, testing

◆ AWARENESS LEVEL

The guide is what the agent knows.
The instructions are how it behaves.
Testing tells us whether either one worked.

From cowork to always-on

1

Assistants

Help when asked

2

Agents

Work toward a defined goal

3

Always-on

Monitor, trigger, act, escalate

AI etiquette

- Good AI use is also professional behavior
- Disclose AI use where appropriate
- Check before recording or summarizing meetings
- Review before sharing externally; do not upload casually
- Keep humans accountable for decisions

Hallucinations and verification

- Fluent does not mean true
- Watch for invented facts and unsupported numbers
- Confident summaries, missing caveats, wrong sources
- Habits: ask for evidence, ask what is missing
- Check the source, use human judgment

◆ THE DANGEROUS KIND

A hallucination is AI wearing a suit and saying something completely wrong with excellent formatting.

Common anti-patterns

- Uploading sensitive data into unmanaged tools
- Treating first drafts as final
- Using Copilot said so as evidence
- Ignoring permissions and oversharing
- Asking broad questions with no source
- Building agents without boundaries

Responsible use in healthcare

- Higher sensitivity means higher discipline
- Patient data, customer-confidential information
- Clinical interpretation, service records, regulated docs
- Use synthetic data for training
- Use approved systems for real work

Demos: agents and trust

- A simple Copilot Studio agent
- Grounded versus ungrounded answers
- Scope leakage: the cafe problem
- Hallucination resistance: not specified

Lunch, then two lab tracks

Track A: business users

- M365 Copilot for work outputs
- Have Word, Outlook, Teams open

Track B: technical / power users

- Copilot Studio agent testing
- Copilot Studio access ready



03

Afternoon: hands-on labs

Two tracks, run in parallel, then a shared workshop

Afternoon labs

- Morning: watch the method. Afternoon: apply it.
- The labs are not a replay of the demos
- Choose audience or output; verify sources
- Test boundaries; improve prompts or agent behavior
- Decide where human judgment is needed

Choose your lab path

Track A: business users

- Role-specific summaries
- Stakeholder reports
- Action trackers
- Safe-use prompts

Track B: technical / power users

- Build a scoped agent
- Test expected behavior
- Expose scope leakage
- Patch and retest boundaries

Apply the method, in two tracks

Track A: work outputs

- A1 Same meeting, different audience
- A2 Build the artifact the stakeholder needs
- A3 Email thread to a comms package
- A4 Your own safe work scenario

Track B: agent testing

- B1 Create base agent, add knowledge
- B2 Happy-path acceptance test
- B3 Boundary and refusal test
- B4 Cafe scope leakage, then criteria

Keep both tracks moving

Track A: prompt pattern

- Goal: what to create
- Context: who for and why
- Source: which uploaded file
- Format: table, email, checklist
- Verification: prove or mark uncertain

Track B: acceptance test

- Purpose and required fields
- When to escalate a fault
- Patient details (should refuse)
- Retention period (not specified)
- A pasta recipe (should refuse)



Coffee break

15 minutes · 14:30 to 14:45

Quality and trust, in two tracks

Track A: verify and improve

- A2.1 Not specified is a good answer
- A2.2 Safe-use decision table
- A2.3 Rewrite one weak prompt

Track B: quality and guardrails

- B2.1 Compact acceptance test
- B2.2 Patch one failure
- B2.3 Agent quality checklist

Verify, then certify

Track A: verification prompts

- Show the exact source text
- If none, say Not specified
- Do not infer or guess

Track B: quality checklist

- Purpose, knowledge source, owner
- Safe refusals and boundaries
- Escalation path and tests
- No pancake problem

Where could an agent help at Topcon?

- Who is the user? What problem does it solve?
- What should the agent do, and not do?
- What knowledge source is needed?
- What risks exist? What governance?
- What is the first small pilot step?



Agent Use Case Canvas

- Use case, user group, problem it solves
- What the agent should do, and must not do
- Knowledge sources; good and refused questions
- Main risks; governance or approval needed
- Human owner; first small pilot step



*What would you like
to see tomorrow?*

Scan the code to share your wish.
We will try to fit them in, but no promises.



Kiitos.

Tomorrow: spec-driven development and GitHub Copilot

Today: Copilot fundamentals, grounding, agents, responsible use

Bring a laptop with VS Code ready; do the Day 2 prerequisites

